

Part 01

Question 1: Why does defining a custom constructor suppress the default constructor?

When a custom constructor is defined, the compiler assumes that the programmer has provided the required initialization logic, so it does not generate a default parameterless constructor automatically.

Question 2: How does method overloading improve readability and reusability?

Method overloading allows multiple methods to have the same name with different parameter lists, making code easier to read and reuse.

Question 3: What is the purpose of constructor chaining in inheritance?

Constructor chaining allows a derived class constructor to call the base class constructor using `base( ... )`, ensuring that inherited members are properly initialized.

Question 4: What is the difference between `new` and `override`?

`new` hides a base class member, while `override` replaces a virtual or abstract member and supports runtime polymorphism.

Question 5: Why should we override `ToString()`?

Overriding `ToString()` provides a meaningful string representation of an object instead of the default representation.

Question 6: Why can't we instantiate an interface?

An interface defines a contract without providing a complete concrete object implementation, so it cannot be instantiated directly.

Question 7: What are the benefits of default interface implementations in C# 8?

They allow interfaces to provide default method implementations, making it easier to add new functionality without requiring all implementing classes to be modified.

Question 8: Why is using an interface reference useful?

It provides abstraction and polymorphism, reduces coupling, and allows code to work with different implementations through the same interface.

Question 9: How does C# overcome single inheritance using interfaces?

Although a class can inherit from only one class, it can implement multiple interfaces, allowing it to support multiple contracts.

Question 10: What is the difference between virtual and abstract methods?

A `virtual` method has a default implementation and can be overridden, while an `abstract` method has no implementation and must be implemented by a concrete derived class.

Part 02

Question 1: What is the difference between a class and a struct in C#?

A class is a reference type, while a struct is a value type. Classes support inheritance, while structs cannot inherit from another class or struct.

Question 2: If inheritance is a relation between classes, what are the other relations between classes?

Other common relationships include association, aggregation, composition, and dependency.